Gaussian Processes

STAT3008 Group 1:
Ethan Jones (44829531), Jay Parfitt (46134006), Angus Scroggie (46387035)

Abstract

As advancements in machine learning are made in recent years, Gaussian processes have reemerged a popular tool for non-parametric modelling. In this report, we will outline the statistical theory behind the Gaussian process, with mention of common kernels and connections to other models. We then motivate the application to time series regression, including discussion of specific modelling considerations and code implementation. Results for two example datasets are presented and analysed before a summary of shortcomings and further topics for study.

Contents

1 Introduction **3**

1.1 Motivation . . . 3

1.2 Preliminaries . . . 3

1.3 Existence of Gaussian Processes . . . 4

1.4 Theory of Covariance Functions and Kernels . . . 5

1.5 Gaussian Process Regression . . . 7

1.5.1 Weight-space view . . . 8

1.5.2 Function-space view . . . 9

1.6 Common Kernels and Key Properties . . . 9

1.7 Connections to Other Models . . . 10

1.7.1 Bayesian linear regression . . . 11

1.7.2 Kernel ridge regression . . . 11

1.7.3 Other models . . . 11

2 Problem Formulation **13**

2.1 Time-series Data . . . 13

2.2 Gaussian Processes for Time-series Analysis . . . 13

2.3 On Kernel Design for Time-series . . . 14

2.4 On Hyperparameter Optimisation . . . 15

3 Implementation **18**

3.1 Overview of Datasets 18

3.2 Overview of Code 20

4 Modelling & Analysis **21**

4.1 Artificial Dataset 21

4.1.1 Modifying the kernel (known) 23

4.2 Practical Dataset – Mean Weekly Temperature in Brisbane 23

4.2.1 Simplifying the kernel (unknown) 25

4.3 Limitations and Improvements 27

5 Conclusion **28**

References **29**

Appendix **31**

1 Introduction

1.1 Motivation

The **non-parametric** framework often stems from limitations of traditional parametric models, such as linear regression. These parametric models require us to explicitly describe each observation as a function of some learned parameters, which can have overbearing assumptions. Non-parametric approaches are flexible, where the data itself shapes the form of the fit. The **Gaussian process** (GP) is a premier model in Bayesian inference, as they define a distribution over all possible functions that the data could be generated from. At a high level, GPs make predictions by incorporating prior knowledge — encoded in the **kernel** (or covariance) function — with the data to produce a posterior distribution. A key advantage of the Bayesian approach is that a posterior offers both a best point estimate (the mean) and uncertainty measurement (the covariance).

This analysis is crucial for tasks like interpolating missing observations and quantifying uncertainty when forecasting. GPs find applications in several domains, such as time series in finance or spatial modelling in geostatistics (where prediction is historically called 'kriging'). Gaussian process theory has existed since at least the 1940s, if not earlier in applied fields like astronomy, but modern computational advancements have made their application widespread. As theory developed, connections were drawn between GPs and many other important models like kernel ridge regression, SVMs, and Bayesian neural networks.

The literature on GPs is vast. In this report we shall be primarily following the notation of Rasmussen & Williams (2006) [6] and introducing many important theorems. For alternate introductions to this topic, we suggest Wang (2023) [21] for a short yet highly accessible tutorial, or Bishop (2006) [7] or Murphy (2022) [20] for a more considered approach.

1.2 Preliminaries

Let us recall some basic probability theory; the Gaussian distribution is of course fundamental to understanding GPs.

Definition 1.1. A random variable X has a **univariate Gaussian distribution** if its probability density function (pdf) is

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2}\frac{(x-\mu)^2}{\sigma^2}\right)$$

with mean μ and variance $\sigma^2 > 0$. This is typically written as $X \sim \mathcal{N}(\mu, \sigma^2)$.

Definition 1.2. A p-dimensional random vector $\boldsymbol{X}$ has a **multivariate Gaussian distribution** if its pdf is

$$p(\boldsymbol{x}) = (2\pi)^{-\frac{p}{2}} |\Sigma|^{-\frac{1}{2}} \exp\left\{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^T \Sigma^{-1} (\boldsymbol{x}-\boldsymbol{\mu})\right\}$$

with now p-dimensional mean vector $\boldsymbol{\mu}$ and $p \times p$ symmetric, positive semi-definite covariance matrix Σ. This is typically written as $\boldsymbol{X} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$.

An important object from this definition is the covariance matrix Σ. For reasons we will see throughout this report, the covariance plays a crucial role in the modelling capabilities of GPs. Each element $\Sigma_{ij} = \mathrm{Cov}(X_i, X_j)$ corresponds to the pairwise covariance of the elements of $\boldsymbol{X}$.

Extending these definitions further, we will see that a **random function** $f(\boldsymbol{X})$ is a probability distribution over the input space $\mathcal{X}$, where realisations are functions themselves. GPs arise from this concept when the function values $\boldsymbol{f}^*$ for a set of inputs $\boldsymbol{x}^*$ form a multivariate

Gaussian distribution. For a much broader mathematical background of the concepts in this report, see Appendix A of Rasmussen & Williams (2006) [6]. This appendix covers fundamentals like marginal and condition distributions; essential Gaussian identities and matrix identities; Cholesky decompositions; and an introduction to measure theory, among other things. For a more theoretical treatment, akin to the exposition of Sections 1.3-1.4, see [5, 10, 13].

1.3 Existence of Gaussian Processes

Before we begin our discussion of why we would possibly employ Gaussian processes, let us first explicate why the very notion makes any sense at all. Consider a simple model. Suppose we reveal data sequentially: first x_1, then x_1, x_2, and so on, under the constraint that each finite collection of observations admits a joint Gaussian density. Initially, we may think of the space of all possible finite Gaussians whose sample means and covariance matrices are consistent with the data revealed thus far. As we continue to reveal more data, this space becomes increasingly restricted. Now imagine revealing data *ad infinitum*: forever. One would hope that, under suitable consistency conditions, these finite Gaussians would converge to a limiting object: an infinite Gaussian, or what we now call a Gaussian process. To understand this limit rigorously, we must first clarify what it means to speak of a stochastic process, and under what conditions an 'infinite Gaussian' can exist.

This section assumes a modest familiarity with measure-theoretic probability, roughly at the level of STAT3004, though the beginning of our exposition proceeds at a level comparable to that of Dudley [5].

Let $\mathcal{X}$ denote an index set. For each finite subset $F = \{x_1, \ldots, x_n\} \subset \mathcal{X}$, define the random vector of function values on F:

$$(f(x_1), \ldots, f(x_n)) : \Omega \to \mathbb{R}^n$$

and its joint law

$$P_F(A) = \mathbb{P}((f(x_1), \ldots, f(x_n)) \in A), \quad A \in \mathcal{B}(\mathbb{R}^n)$$

that is, the pushforward measure of $\mathbb{P}$ under the evaluation map $\omega \mapsto (f(x_1)(\omega), \ldots, f(x_n)(\omega))$. When the process f is Gaussian, each finite-dimensional law is multivariate normal:

$$P_F = \mathcal{N}(m_F, K_F)$$

where $(m_F)_i = m(x_i)$ and $(K_F)_{ij} = k(x_i, x_j)$ for some functions $m : \mathcal{X} \to \mathbb{R}$ and $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$. We now define the notion of consistency.

Definition 1.3 (Consistency)**.** A family $\{P_F : F \subset \mathcal{X}, |F| < \infty\}$ is said to be consistent if whenever $F \subseteq G \subset \mathcal{X}$ and $|G| < \infty$, we have

$$P_F = P_G \circ \pi_{F,G}^{-1}$$

where $\pi_{F,G} : \mathbb{R}^G \to \mathbb{R}^F$ is the canonical projection.

In other words, the family $\{P_F\}$ is consistent if the marginal distribution on any smaller index set F agrees with the restriction of the larger one G. Having formalised the notion of consistency, we can now ask whether every such coherent family of finite-dimensional distributions arises as the system of marginals of some probability measure on the product space. The Kolmogorov Extension Theorem gives an affirmative answer: any consistent family uniquely determines a probability measure $\mathbb{P}$ whose finite-dimensional coincide with the given P_F.

Intuitively, the theorem ensures that if all finite collections of random variables agree where they overlap, then there exists an underlying function

$$f : \mathcal{X} \to \mathbb{R}$$

whose behaviour on every finite subset matches those specifications. In the Gaussian case, this means that the joint behaviour of any finite set of points already determines the law of the entire

process. This is the idea that underlies Gaussian process regression. We now state the theorem:

Theorem 1 (Kolmogorov Extension Theorem; Dudley 12.1.2)**.** *Let $(S_t, \mathcal{B}_t)_{t\in\mathcal{X}}$ be measurable spaces, each universally measurable (in particular, $S_t = \mathbb{R}$ with the Borel σ-algebra). For any consistent family of finite-dimensional laws $\{P_F : F \subset \mathcal{X}\}$, there exists a unique probability measure*

$$\mathbb{P} : \mathcal{B}(\mathbb{R})^{\otimes\mathcal{X}} \to [0, 1]$$

on the product measurable space $(\mathbb{R}^{\mathcal{X}}, \mathcal{B}(\mathbb{R})^{\otimes\mathcal{X}})$ such that

$$\mathbb{P} \circ \pi_F^{-1} = P_F, \quad \forall F \subset \mathcal{X} \text{ (F finite)}$$

That is, $\mathbb{P}$ has precisely the given finite-dimensional marginals.

An immediate corollary of the Kolmogorov Extension Theorem is the Gaussian case:

Theorem 2 (Gaussian Case (Existence of GPs); Dudley 12.1.3)**.** *Let $\mathcal{X}$ be an index set, $m : \mathcal{X} \to \mathbb{R}$ a mean function, and $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ a symmetric, positive semidefinite kernel. Then there exists a stochastic process*

$$f : \mathcal{X} \to \mathbb{R}$$

such that for every finite subset $\{x_1, \ldots, x_n\} \subseteq \mathcal{X}$,

$$(f(x_1), \ldots, f(x_n)) \sim \mathcal{N}(m, K)$$

where $m_i = m(x_i)$ and $K_{ij} = k(x_i, x_j)$.

By the theorem above, any choice of mean $m : \mathcal{X} \to \mathbb{R}$ and symmetric, positive semi-definite covariance $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ defines a Gaussian process $f \sim \mathcal{G}P(m, k)$. In particular, if for every finite subset $F = \{x_1, \ldots, x_n\} \subset \mathcal{X}$ the matrix

$$K_F = [k(x_i, x_j)]_{i,j=1}^n$$

is positive semi-definite, then there exists a Gaussian process whose finite-dimensional marginal satisfies

$$(f(x_1), \ldots, f(x_n)) \sim \mathcal{N}(m_F, K_F)$$

where $(m_F)_i = m(x_i)$ for $i = 1, \ldots, n$. Because a multivariate law is determined by its mean and covariance, the pair (m, k) uniquely determines the finite-dimensional laws of f, and hence its process law.

1.4 Theory of Covariance Functions and Kernels

In this section, we reinterpret covariance functions, commonly called kernels, as the central objects that specify both the geometry and inductive bias of a Gaussian process. To quote Rasmussen & Williams: "Under the Gaussian process view, it is the covariance function that defines nearness or similarity" [6].

The section on the existence of Gaussian processes established, following the Gaussian specialisation of Kolmogorov's Extension Theorem, see Theorem 12.1.3 of Dudley [5], that a Gaussian process exists whenever its covariance function k is symmetric and positive semidefinite. In what follows, we examine how these properties ensure that k defines a valid covariance structure—and, equivalently, a kernel—and how the choice of k controls the expressiveness and behaviour of Gaussian process models.

So far we have spoken of positive definite covariance functions and kernels, assuming the reader's familiarity. For completeness, we now make this notion precise.

Definition 1.4 (Positive Definite Kernels)**.** Let $\mathcal{X}$ be a non-empty set. A symmetric function $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is called a positive definite kernel if for every finite subset $\{x_1, \ldots, x_n\} \subset \mathcal{X}$ and

every real vector $(c_1, \ldots, c_n) \in \mathbb{R}^n$,

$$\sum_{i=1}^{n} \sum_{j=1}^{n} c_i c_j k(x_i, x_j) \geq 0$$

Equivalently, k is positive semidefinite if and only if the Gram matrix

$$K = [k(x_i, x_j)]_{i,j=1}^{n}$$

is positive semidefinite for every finite collection $(x_1, \ldots, x_n) \in \mathcal{X}^n$, i.e. $a^T K a \geq 0$ for all $a \in \mathbb{R}^n$. By the Moore-Aronszajn theorem,[1]

$$k(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{H}}$$

In this sense, the kernel encodes a geometry (a similarity): just as a Euclidean inner product measures how much one vector lies in the direction of another, k defines distances and angles in the implicit feature space. Two inputs with large $k(x, x')$ have feature vectors with a small angle (high cosine similarity), so under the GP prior $f(x)$ and $f(x')$ are strongly correlated.

The kernel therefore shapes the hypothesis class: it determines which functions are *a priori* likely. For instance, the squared-exponential (RBF) kernel corresponds to an infinite-dimensional feature mapping[2] and enforces extremely smooth sample paths. Shorter length-scales treat points as 'near' only when very close in $\mathcal{X}$, yielding rougher functions, whereas longer length-scales induce smoother — in a sense, 'global' — correlations.

To formalise this intuition, see chapter 4 of Rasmussen & Williams (2006) [6], view k as an integral operator on $L^2(\mathcal{X}, \mu)$:

$$(T_k f)(x) = \int_{\mathcal{X}} k(x, x') f(x') d\mu(x')$$

If k is continuous (thus bounded) on compact $\mathcal{X}$, then k is Hilbert-Schmidt and hence T_k is compact[3] and self-adjoint. Then, by the spectral theorem (see theorem 5.1 of Conway's Functional Analysis [8]), T_k admits an orthonormal eigenbasis in L^2, which yields Mercer's expansion.

Theorem 3 (Mercer's Theorem [1])**.** *If k is continuous, symmetric, and p.s.d. on compact $\mathcal{X}$, then there exist $\lambda_i \geq 0$ and an orthonormal eigenbasis $\{\phi_i\}_{i=1}^{\infty} \subset L^2(\mathcal{X}, \mu)$ such that*

$$k(x, x') = \sum_{i=1}^{\infty} \lambda_i \, \phi_i(x) \phi_i(x')$$

with absolute and uniform convergence on $\mathcal{X} \times \mathcal{X}$.

This result makes precise the idea that a kernel behaves like an inner product in a (possibly infinite-dimensional) feature space. Each eigenfunction ϕ_i represents a principal direction (mode of variation to be precise), and the corresponding eigenvalue λ_i quantifies how much covariance or 'spectral weight' lies along that direction. In this sense, Mercer's theorem reveals structure implicit in k: it decomposes the covariance into principal components (which are orthogonal), just as an eigen-decomposition does for finite covariance matrices.

[1] The Moore-Aronszajn theorem guarantees that every symmetric, positive semidefinite kernel defines a unique reproducing kernel Hilbert space (RKHS) $\mathcal{H}$ such that $k(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{H}}$.

[2] e.g., via its Mercer expansion, we use this implicitly to justify calling the feature space "possibly infinite-dimensional."

[3] See Theorem 8.83 of [9] for a proof that the integral operator $T_k \in \mathcal{L}(L^2(\mathcal{X}, \mu))$ associated with a Hilbert-Schmidt kernel is compact, Lemma 8.20 of [9]

This spectral perspective also makes the connection to Gaussian processes more clear, as we will see. With $k(x, x') = \sum_i \lambda_i \phi_i(x)\phi_i(x')$, define the stochastic process

$$f(x) = \sum_{i=1}^{\infty} w_i \, \phi_i(x), \qquad w_i \overset{\text{iid}}{\sim} \mathcal{N}(0, \lambda_i)$$

Then $\mathbb{E}[f(x)] = 0$ and

$$\begin{aligned}
\mathrm{Cov}[f(x), f(x')] &= \mathbb{E}[(f(x) - \mathbb{E}[f(x)])(f(x') - \mathbb{E}[f(x')])] \\
&= \mathbb{E}[f(x)f(x')] \\
&= \mathbb{E}\left[\sum_i \sum_j w_i w_j \phi_i(x)\phi_j(x')\right] \\
&= \sum_i \sum_j \mathbb{E}[w_i w_j]\phi_i(x)\phi_j(x')
\end{aligned}$$

Since w_i, w_j are independent for $i \neq j$ and are zero-mean, non-diagonal terms go to zero. If $i = j$, we have $\mathbb{E}[w_i w_j] = \mathbb{E}[w_i^2] = \mathrm{Var}(w_i) = \lambda_i$. Hence,

$$\mathrm{Cov}[f(x), f(x')] = \sum_i \lambda_i \phi_i(x)\phi_i(x') = k(x, x')$$

where the final equality is due to Mercer's theorem. Thus $f \sim \mathcal{GP}(0, k)$.[4]

Finally, we can connect this work to the results in Section 1.3; Theorem 2 ensures that any symmetric p.s.d k yields a Gaussian process. Dudley's Theorem 12.1.4 (isonormal processes) [5] realises this GP as a linear Gaussian functional L on a Hilbert space $\mathcal{H}$ with $\mathbb{E}[L(h)L(g)] = \langle h, g\rangle_{\mathcal{H}}$. Mercer's theorem identifies $\mathcal{H}$ with the feature map, if:

$$k(x, x') = \sum_i \lambda_i \phi_i(x)\phi_i(x')$$

then $\phi(x) = (\sqrt{\lambda_1}\phi_1(x), \sqrt{\lambda_2}\phi_2(x), \dots)$ [13] and

$$f(x) = L(\phi(x)) = \sum_i w_i \phi_i(x), \quad w_i \sim \mathcal{N}(0, \lambda_i)$$

that is, the Gaussian process is the isonormal process on the RKHS of k.[5]

1.5 Gaussian Process Regression

We now step back from the highly mathematical approach of the previous sub-section to the practical formulation used in regression and restate, officially, the definition of a Gaussian Process:

Definition 1.5 (Gaussian Process (Non-Measure-Theoretic))**.** A stochastic process $f = \{f(x) : x \in \mathcal{X}\}$ is called a Gaussian process if for every finite subset $F = \{x_1, \dots, x_n\} \subset \mathcal{X}$:

$$(f(x_1), \dots, f(x_n)) \sim \mathcal{N}(m_F, K_F)$$

where $m_F = (m(x_i))_{i=1}^n$ is the restriction of some mean function $m : \mathcal{X} \to \mathbb{R}$ to F and $(K_F)_{ij} = k(x_i, x_j)$ is the restriction of a symmetric, positive semidefinite covariance function (or kernel)

[4] Convergence is in mean-square. We implicitly invoke the KL theorem to pass from the Mercer expansion of k to a convergent stochastic series for f [1].

[5] Moore-Aronszajn gives the RKHS and Dudley 12.1.4 gives the isonormal process, identifying ϕ as above links them via the Mercer basis.

$k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ to $F \times F$.

This definition follows immediately from the Gaussian specialisation of the Kolmogorov Extension Theorem (Section 1.3 Theorem 2). Namely, for any mean function $m : \mathcal{X} \to \mathbb{R}$ and any symmetric, positive semidefinite covariance function $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$, there exists a Gaussian process $f \sim \mathcal{GP}(m, k)$. This particular formulation allows us to model our data as a Gaussian process in GP regression.

We now introduce GP regression. Fix a mean $m : \mathcal{X} \to \mathbb{R}$ and a symmetric p.s.d kernel $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$, theorem 2 of section 1.3 guarantees a process $f \sim \mathcal{GP}(m, k)$. In regression we observe noisy targets

$$y_i = f(x_i) + \varepsilon_i, \quad \varepsilon_i \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma_n^2)$$

our task is to infer the posterior distribution over functions given data X, y. We will now demonstrate that GP regression may be understood as the infinite-dimensional limit of a Bayesian linear model, that is, the weight-space view of Rasmussen & Williams (2006) [6].

1.5.1 Weight-space view

Note that in this section, we more or less summarise the results of Rasmussen & Williams work [6, §2.1], taking advantage of many theorems. Having established the existence of Gaussian processes through theorems shown in section 1.3 and identified their structure via Mercer's theorem 1.4 and the isonormal construction [5], we now consider a finite-dimensional realisation that motivates their use in regression, the weight-space view.

Intuitively, the weight-space model can be seen as the finite-dimensional analogue of the isonormal process discussed earlier. There, each random function $f(x) = L(\phi(x))$ was obtained by applying a Gaussian linear functional L to a feature map $\phi(x)$ in a Hilbert space. In this section, we make this construction explicit in finite coordinates.

As per chapter 2.1 Rasmussen & Williams (2006) [6], consider a training set $D = \{(x_i, y_i)\}_{i=1}^n$ where $x_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$. We begin with a linear model of the form

$$f(x) = x^T w, \quad y = f(x) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma_n^2)$$

We place a Gaussian prior over the weights,

$$w \sim \mathcal{N}(0, \Sigma_p)$$

encoding our prior uncertainty about the coefficients. This yields the likelihood $p(y|X, w) = \mathcal{N}(y|X^T w, \sigma_n^2 I)$ and posterior

$$p(w|X, y) = \mathcal{N}(w|\bar{w}, A^{-1}), \quad A = \sigma_n^{-2} X X^T + \Sigma_p^{-1}, \quad \bar{w} = \sigma_n^2 A^{-1} X y$$

The predictive distribution for a new input x_* is Gaussian:

$$p(f_*|x_*, X, y) = \mathcal{N}(x_*^T \bar{w}, x_*^T A^{-1} x_*)$$

As explicated in §2.1 of Rasmussen and Williamson, this model, ridge regression, provides the maximum-a-posteriori (MAP) estimate for w a $\bar{w}$, yielding a predictive mean and variance consistent with Bayesian uncertainty. See Section 1.7.2 for more details.

To increase flexibility, the inputs may be mapped to a feature space via a fixed transformation: $\phi : \mathcal{X} \to \mathbb{R}^M$, yielding

$$f(x) = \phi(x)^T w, \quad w \sim \mathcal{N}(0, \Sigma_p)$$

Following Rasmussen (§2.1.2), the predictive distribution becomes

$$p(f_* \mid x_*, X, y) = \mathcal{N}\big(\phi(x_*)^\top \Sigma_p \Phi [K + \sigma_n^2 I]^{-1} y,\ \phi(x_*)^\top \Sigma_p \phi(x_*) - \phi(x_*)^\top \Sigma_p \Phi [K + \sigma_n^2 I]^{-1} \Phi^\top \Sigma_p \phi(x_*)\big)$$

where

$$K_{ij} = k(x_i, x_j) = \phi(x_i)^\top \Sigma_p \phi(x_j), \qquad \Phi = \begin{bmatrix} \phi(x_1)^\top \\ \vdots \\ \phi(x_n)^\top \end{bmatrix}, \qquad K = \Phi \Sigma_p \Phi^\top$$

As noted in Rasmussen, this establishes that the kernel can be regarded as the inner product of feature representations under the metric induced by Σ_p. From this point, the kernel becomes the fundamental object: any algorithm expressible solely in terms of these inner products can be formulated implicitly in feature space, a principle known as the kernel trick.

1.5.2 Function-space view

An equivalent perspective is obtained by starting directly with a distribution over functions, rather than over weights. A Gaussian process is formally defined as a collection of random variables such that any finite subset follows a joint Gaussian distribution:

$$f \sim \mathcal{GP}(m, k)$$

where

$$m(x) = \mathbb{E}[f(x)], \qquad k(x, x') = \mathrm{Cov}[f(x), f(x')]$$

This definition automatically satisfies the consistency property guaranteed by the Kolmogorov Extension Theorem (cf. Theorem 2), ensuring that the finite–dimensional marginals are coherent.

As shown in §2.2 of Rasmussen & Williams the Bayesian linear model described earlier is already a Gaussian process:

$$\mathbb{E}[f(x)] = 0, \qquad \mathbb{E}[f(x)f(x')] = \phi(x)^\top \Sigma_p \phi(x') = k(x, x')$$

Hence, the function values $(f(x_1), \ldots, f(x_n))$ are jointly Gaussian, and the covariance function k encodes correlations between outputs as a function of input similarity.

Given training inputs X and noise–corrupted targets

$$y = f(X) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma_n^2 I),$$

the joint prior over y and test function values f_* is

$$\begin{bmatrix} y \\ f_* \end{bmatrix} \sim \mathcal{N}\left(0, \begin{bmatrix} K(X,X) + \sigma_n^2 I & K(X, X_*) \\ K(X_*, X) & K(X_*, X_*) \end{bmatrix}\right)$$

Conditioning gives the posterior predictive distribution

$$f_* \mid X, y, X_* \sim \mathcal{N}\big(\bar{f}_*, \, \mathrm{cov}(f_*)\big)$$

where

$$\bar{f}_* = K(X_*, X)\,[K(X,X) + \sigma_n^2 I]^{-1} y \qquad \mathrm{cov}(f_*) = K(X_*, X_*) - K(X_*, X)\,[K(X,X) + \sigma_n^2 I]^{-1} K(X, X_*)$$

This is identical to the predictive distribution derived in the weight–space formulation (Rasmussen and Williams Eq 2.2) establishing their equivalence.

1.6 Common Kernels and Key Properties

With the theoretical heavy-lifting of GPs and kernels outlined above, we can solidify these concepts with some examples of common kernels and important properties. There are several more kernel options than mentioned in this section, but the kernels below are relevant for our time series analysis later. Refer to Duvenaud's *Kernel Cookbook* [25] for more related discussion.

Perhaps the most common kernel is the radial basis function (RBF) or squared-exponential (SE) kernel:

$$k(x, x') = \zeta^2 \exp\left(-\frac{|x - x'|^2}{2l^2}\right) \tag{1}$$

where l is called the *lengthscale* and controls the smoothness (or wigglyness) of the function. All kernel functions have a scale factor ζ^2 out the front, which controls the vertical magnitude of the function away from the mean. Related is the rational quadratic

$$k(x, x') = \zeta^2 \left(1 + \frac{|x - x'|^2}{2\alpha l^2}\right)^{-\alpha} \tag{2}$$

which can be intuited as adding multiple RBF kernels together to vary smoothly across multiple lengthscales; α controls the weighting of these variations.

Another kernel family is periodic kernels, commonly written

$$k(x, x') = \zeta^2 \exp\left(-\frac{2\sin^2(\pi\,|x - x'|\,/p)}{l^2}\right) \tag{3}$$

with period p. As we will see when modelling time series, these kernels are crucial and can be further modified as required. The kernels considered so far are called *stationary* since they are functions of the difference $|x - x'|$. A common *non-stationary* kernel is the linear (or dot-product) kernel,

$$k(x, x') = \zeta^2 \left(c + x \cdot x'\right) \tag{4}$$

which namely uses the dot product $x \cdot x'$ to measure distance, where c controls the inhomogeneity. As expected, using this kernel in a GP corresponds to Bayesian linear regression, as mentioned in the next section. Location is hence important for the dot-product kernel, and we can exponentiate it to model polynomial trends instead.

This leads us to a highly essential property of kernels: they are closed under addition and multiplication. Performing *composition* of basic kernels allows us to specify priors for much more complex phenomena, like time series behaviour (Section 2.3). In fact, this is why we can multiply by the scalars ζ^2. Furthermore, the concept of a kernel can be extended to non-numeric input spaces, like text or images, as long as the main properties like symmetry and positive semi-definiteness hold. This allows us to do inference in any space that can define the notion of 'distance', which is a significant result of kernel theory in general.

The choice of covariance function is very important, as it encodes our prior beliefs about the function we want to model. As mentioned by Roberts et al. (2013), "the covariance forms the beating heart of GP inference." One must hence be careful if choosing an RBF kernel for its popularity alone. This kernel is stationary, so extrapolating outside the data range means predictions will eventually fall to zero, which may not reflect the intended behaviour of the true function. What exactly defines 'nearness' in a GP is a critical choice of the user.

1.7 Connections to Other Models

Gaussian processes unify a range of modelling paradigms in statistics and machine learning. They generalise Bayesian linear regression, connect to kernel ridge regression and stochastic processes such as Brownian motion, and link conceptually to infinite neural networks and other kernel-based or state-space models.

1.7.1 Bayesian linear regression

Bayesian linear regression is a finite-dimensional special case of a Gaussian process prior. Consider the model

$$y = f(x) + \varepsilon, \quad f(x) = \boldsymbol{w}^\top \phi(x), \quad \boldsymbol{w} \sim \mathcal{N}(0, \Sigma_{\boldsymbol{w}}), \quad \varepsilon \sim \mathcal{N}(0, \sigma_y^2)$$

Integrating over the Gaussian weight prioe gives the marginal covariance

$$\kappa(x, x') = \phi(x)^\top \Sigma_{\boldsymbol{w}} \phi(x')$$

so predictions in weight space and in function space become equivalent. When $\phi(x) = x$ and $\Sigma_{\boldsymbol{w}} = \sigma_{\boldsymbol{w}}^2 \boldsymbol{I}$, the kernel reduces to the linear form $\kappa(x, x') = \sigma_{\boldsymbol{w}}^2 x^\top x'$, making Gaussian process regression identical to Bayesian linear regression [20]. Hence, Bayesian linear regression can be viewed as a Gaussian process with a finite feature basis, while Gaussian processes extend this to infinite-dimensional feature mappings through the kernel.

1.7.2 Kernel ridge regression

For a zero mean Gaussian process prior $f \sim \mathcal{GP}(0, k)$ and Gaussian observation noise $\varepsilon \sim \mathcal{N}(0, \sigma^2)$, the predictive mean at test inputs X_* is

$$\hat{f}(X_*) = K_{X_* X} \left(K_{XX} + \sigma^2 \boldsymbol{I}\right)^{-1} y$$

which is mathematically equivalent to the kernel ridge regression estimator with regularisation parameter $\lambda = \sigma^2$. Both methods minimise the same regularised least-squares objective

$$\|y - K\alpha\|^2 + \lambda\, \alpha^\top K \alpha$$

but the Gaussian process formulation provides a full posterior distribution over functions rather than a single point estimate. The equivalence becomes clear when viewing ridge regression in its dual form [4]. Their derivation expresses predictions purely in terms of kernel evaluations,

$$\hat{y} = y^\top (K + \lambda \boldsymbol{I})^{-1} k$$

where k is the vector of kernel values between training and test points. This dual representation corresponds exactly to Gaussian process inference with a Gaussian prior over functions and noise variance $\sigma^2 = \lambda$.

The dual ridge regression view was also motivated by overcoming the *curse of dimensionality* through kernel functions, allowing linear models to operate in implicit high-dimensional feature spaces without explicit computation of basis expansions [4]. From a probabilistic perspective, the same structure arises naturally in Gaussian processes, where the kernel encodes prior covariance between function values, and the regularisation term corresponds to observational noise.

Hence, the MAP estimate under a Gaussian process prior coincides exactly with the kernel ridge regression solution, while the Gaussian process posterior further captures predictive uncertainty through its covariance function.

1.7.3 Other models

Gaussian processes can also be understood as generalisations of classical stochastic processes. A key example is **Brownian motion**, which is itself a Gaussian process with covariance function $k(t, s) = \min(t, s)$. This expresses that function values evolve smoothly over time, with uncertainty growing in proportion to elapsed time. Integrated forms of Brownian motion, sometimes called *Wiener process priors*, underlie spline models, where smoothness constraints emerge naturally from the process definition. Thus, familiar tools such as smoothing splines can be seen as special cases of Gaussian process regression with specific kernel choices.

Beyond these temporal processes, **connections to neural networks** arise when considering infinitely wide Bayesian neural networks. As the width of a hidden layer grows without bound, the network's prior over functions converges to a Gaussian process, with the kernel determined by the network's activation function and weight priors [7] [20]. This perspective shows that Gaussian processes and neural networks are not fundamentally separate but exist on a continuum of models differing mainly in parametrisation.

Gaussian processes also share close ties with several other model families, with a few notable mentions listed below.

- **Radial basis function (RBF) networks:** finite basis approximations to Gaussian process models [7] [20].
- **Splines:** equivalent to integrated Wiener process priors that impose smoothness [6].
- **Kalman filters and state–space models:** exact inference for certain Gaussian processes with time-dependent (e.g., Matérn) kernels [6] [20].
- **Support vector machines and other kernel methods:** share the same underlying reproducing kernel Hilbert space (RKHS) framework, but themselves are parametric models [6] [4].
- **Gaussian Markov random fields:** sparse approximations to continuous Gaussian processes on discrete grids [11].

Together, these relationships reveal that Gaussian processes unify concepts from regression, kernel methods, stochastic processes, and neural networks within a single probabilistic framework. With these connections in mind, it is clear that the Gaussian process is a fundamental and flexible non-parametric model. For further discussion and mathematical insights into these relationships with other models, see Chapter 6 of Rasmussen & Williams (2006) [6].

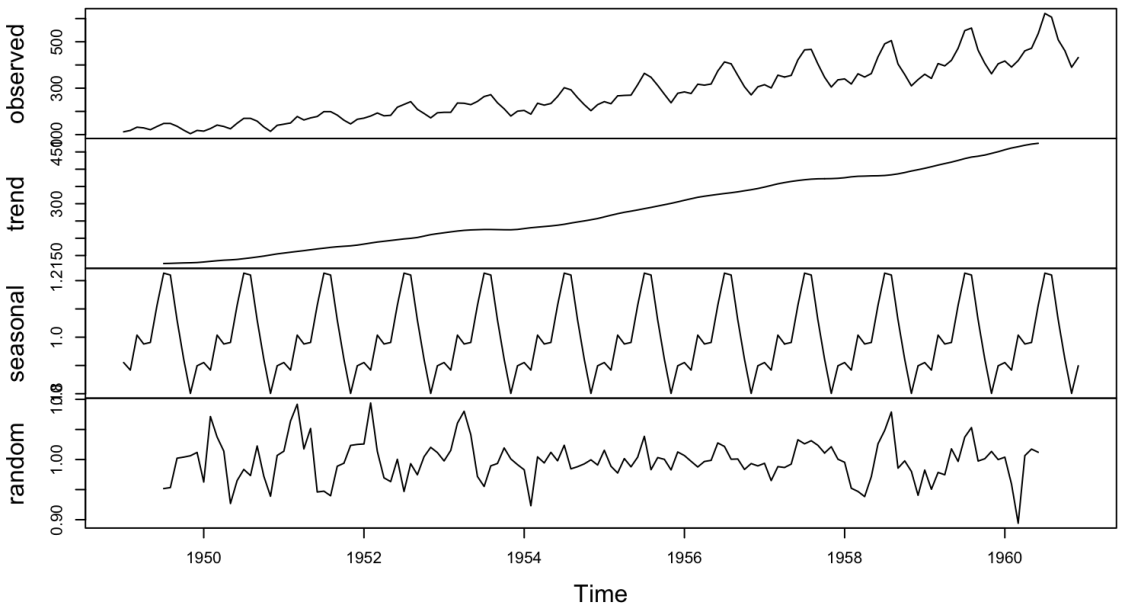

Figure 1: A visual example of a classical time series decomposition. Image taken from [16].

2 Problem Formulation

2.1 Time-series Data

A time series is simply a dataset that is indexed by time. Such observations are often recorded sequentially in discrete time and may be written as $\mathcal{D} = \{t_i, y_i\}_{i=1}^{T}$. They are ubiquitous in modern data analytics, with some common examples of time series applications being environmental monitoring (weather observations, water levels) and financial markets (stock prices). An analyst may typically be interested in curve-fitting (interpolation) or forecasting trends (extrapolation), for which there are a variety of models at their disposal. Filtering and smoothing signal can be achieved with techniques like autocorrelation analysis and trend decompositions.

Time series are usually described as consisting of three main components, shown in Figure 1. The **trend** is an underlying, long-term movement in the average over time. **Periodicity** is encoded in either seasonality (fixed periods) or the more general notion of cyclic variations (non-fixed fluctuations). Lastly, **noise** describes the irregularities in measurements which the other components do not account for; this is usually eliminated by smoothing with a rolling average or similar.

Conducting such decomposition and forecasting are important for several reasons. Of course, accurately predicting future trends is crucial for strategic planning and risk management, and affects domains like public health and finance. Furthermore, modelling periodic patterns in time is essential for identifying anomalies like natural disasters. Hence, the need for flexible models which also quantify uncertainty in all these components makes Gaussian processes quite attractive.

2.2 Gaussian Processes for Time-series Analysis

Based on this general decomposition of time series, one is highly allured by the elegance of the Bayesian non-parametric approach. This approach is desirable for time series analysis because it offers implicit regularisation and an inbuilt handling of uncertainty across predictions. In particular, GPs operate within function space itself and leave us the freedom to specify prior beliefs on the covariance behaviour. This point on kernel choices will be examined shortly.

The ability for GPs to encode these complex assumptions into a prior, while simultaneously allowing the model to fit flexibly as a random function, is a great example of Occam's razor in statistical modelling. Unlike traditional parametric approaches to time series analysis, such as ARIMA or NNs, GPs provide us with the full posterior distribution over functions for more robust forecasting capabilities with uncertainty estimates.[6] Using the machinery outlined in the previous section, we shall hence demonstrate and discuss the application of Gaussian processes for regression on (univariate) time series data. In the remainder of this report, we focus on considerations for kernel design, hyperparameter optimisation, analysis, and forecasting when fitting a GP to a time series.

Consider analysing a time series as a regression problem

$$y = f(t) + \varepsilon$$

where $t \in \mathbb{R}$, $f : \mathbb{R} \to \mathbb{R}$, and $\varepsilon \sim N(0, \sigma^2)$ is the iid noise. We therefore assume that f is modelled by a Gaussian process

$$f \sim \mathcal{GP}(0, k_{\boldsymbol{\theta}})$$

where $k_{\boldsymbol{\theta}} : \mathbb{R} \times \mathbb{R} \to \mathbb{R}$ is our chosen covariance function with hyperparameters $\boldsymbol{\theta}$. Note that we have chosen the mean function to be zero, as in general we have no belief *a priori* about the trend falling above or below the average.

2.3 On Kernel Design for Time-series

Kernel design is not an objective procedure, but requires (prior) knowledge of the problem domain and oftentimes some type of trial-and-error. It is for this reason that many Bayesian modelling experts warn against using 'default' kernels in code implementations, which is usually the RBF kernel [25]. Many resources examine the construction of GP kernels; see [6, 25] in general and [11, 17] for time series applications. Nonetheless, we outline the framework and summarise the relevant ideas for time series here.

As established in 1.6, the kernel specifies our prior belief about the function space used for curve fitting. Furthermore, positive definiteness holds under addition and multiplication of kernels, so those basic kernels become building blocks for more complex covariance structures. Hence, we can represent our belief of typical time series elements by constructing a suitable kernel via composition. A simple kernel for time series modelling may look like

$$k_{\boldsymbol{\theta}}(x, x') = k_{\text{TRE}}(x, x') + k_{\text{PER}}(x, x') + k_{\text{IRR}}(x, x') + k_{\text{WN}}(x, x')$$

where

- the long-term trend k_{TRE} could be modelled by a linear or RBF kernel;
- the seasonality k_{PER} is modelled by a valid periodic kernel;
- any irregular cyclic variations k_{IRR} could be modelled by a rational quadratic or Matérn kernel; and
- noise k_{WN} is modelled by a white noise kernel.

It is important to stress that this example composition is by no means definitive. A very common extension is to *quasi*-periodic kernels by multiplying a periodic with a basic stationary kernel like RBF. Hence, the latter component now describes the rate of evolution of the periodic signal,

[6] Additionally, many of these models can be constructed as limiting cases of Gaussian processes with specific kernel choices! Mentioned before were AR models and NNs, but indeed linear models and even GAMs can fall within the GP framework. See 1.7 and [7] 6.4.7 and [6] Ch. 6.

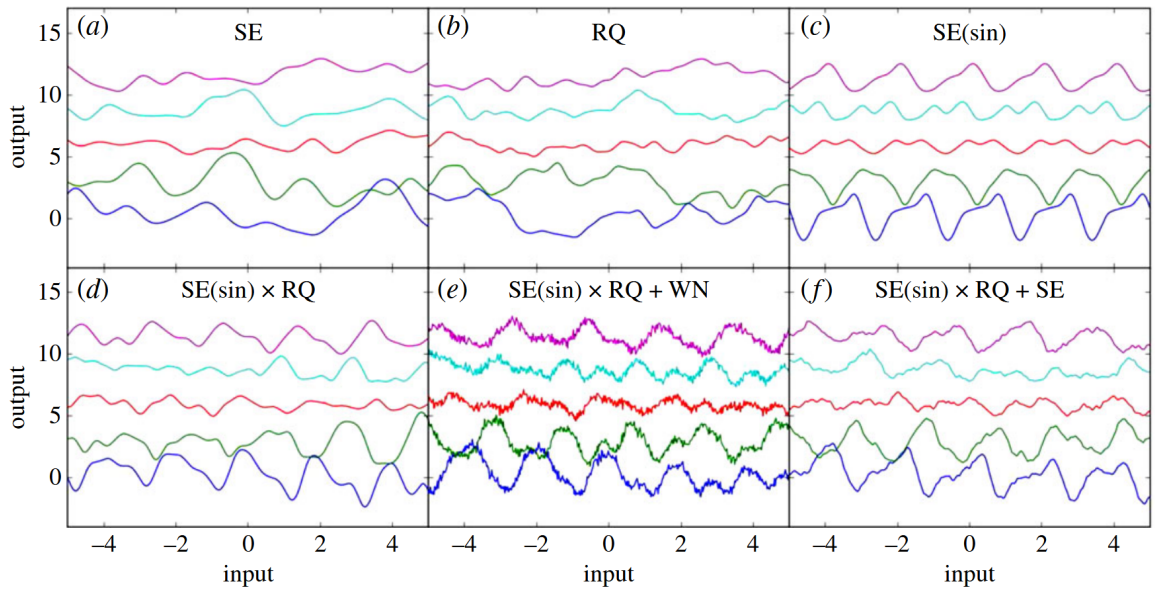

Figure 2: Randomly sampled functions based on different kernels. In particular, the bottom row shows some examples of quasi-periodicity which is crucial for realistic modelling of time series. Image taken from [11].

for instance, allowing decay away from rigid periodicity over time.

$$k_{\text{PER}}(x, x') = \zeta^2 \exp\left(-\frac{2\sin^2(\pi\,|x - x'|\,/p)}{l_1^2}\right) \exp\left(-\frac{|x - x'|^2}{2l_2^2}\right) \tag{5}$$

Some visual examples of this are shown in Figure 2. Another example is a noise component that accounts for correlation using another RBF; perhaps the time series has a double seasonality and so requires two independent quasi-periodic kernels, and so on. Some developments in the literature propose automatic selection and optimisation of kernels by *automatic relevance determination* (ARD). See, for instance, Corani et al. (2021) [17] who have some convincing results utilising a robust kernel composition with spectral mixture kernels in conjunction with ARD. See also the PhD thesis by Duvenaud (2014) [12] which covers automatic model construction and description using GPs.[7]

2.4 On Hyperparameter Optimisation

The last item to cover before some practical analyses is hyperparameter optimisation for GPs. The main problem is that while GPs are indeed non-parametric models, we do have the parameters $\boldsymbol{\theta}$ that specify the covariance function $k_{\boldsymbol{\theta}}$. For example, we have seen that $\boldsymbol{\theta}$ could contain some lengthscale l, vertical scaling ζ^2, and noise variance σ^2. While we can choose desired covariance structures quite easily, hyperparameter selection within specific kernels is not so straightforward. This is crucial for both good curve fitting and generalisation, especially if we hope to forecast for a time series. Again, many of the references in this report give excellent overviews of this tuning problem and various methods to approach it. In particular, one is directed to Chapter 5 of Rasmussen & Williams (2006) [6] or Section 17.2.6 of Murphy (2022) [20] for comprehensive mathematical treatments, or Section 4 of Roberts et al. (2013) [11] for a more intuitive explanation.

A standard approach to surveying hyperparameter space in machine learning is to perform

[7]Duvenaud being the author of the *Kernel Cookbook* [25], who furthermore had Rasmussen as their PhD supervisor — one of the authors in the seminal text *Gaussian Processes for Machine Learning* [6] — both of which were crucial references for this project.

a grid search with cross-validation (CV). However, the size of $\boldsymbol{\theta}$ increases as the kernel gets more complex, as seen in our case with time series analysis. Notably, non-probabilistic kernel methods like SVMs must take this slow approach. Rasmussen & Williams (2006) [6] discuss a more appealing gradient-based LOO-CV procedure which is faster, and modern approaches like genetic algorithms are feasible.

These methods aside, we are motivated to instead leverage the probabilistic framework of GPs, as explained by Rasmussen & Williams (2006) [6]:

> "Bayesian principles provide a persuasive and consistent framework for inference. Unfortunately, for most interesting models for machine learning, the required computations (integrals over parameter space) are analytically intractable, and good approximations are not easily derived. Gaussian process regression models with Gaussian noise are a rare exception: integrals over the parameters are analytically tractable and at the same time the models are very flexible."

As suggested, an *empirical Bayes* approach provides the option to maximise the (log-)marginal likelihood (LML) to obtain the optimal parameters:

$$\boldsymbol{\theta}^{\star} = \arg\max_{\boldsymbol{\theta}} \log p\left(\boldsymbol{y} \mid X, \boldsymbol{\theta}\right)$$

With the convenience of all distributions in the GP framework being Gaussian, the marginal likelihood is shown to be $p\left(\boldsymbol{y} \mid X, \boldsymbol{\theta}\right) = N(\boldsymbol{y} \mid \mathbf{0}, \tilde{K})$ and thus

$$\log p\left(\boldsymbol{y} \mid X, \boldsymbol{\theta}\right) = -\frac{1}{2}\boldsymbol{y}^T \tilde{K}^{-1}\boldsymbol{y} - \frac{1}{2}\log|\tilde{K}| - \frac{n}{2}\log 2\pi \tag{6}$$

where $\tilde{K} = K + \sigma^2 I$ is the noisy covariance matrix, which has an implicit dependence on $\boldsymbol{\theta}$. The expression consists of three terms:

1. The data fit term $-\frac{1}{2}\boldsymbol{y}^T \tilde{K}^{-1}\boldsymbol{y}$ penalises deviations from the observations, similar to squared error, to reduce model bias.
2. The complexity term $-\frac{1}{2}\log|\tilde{K}|$ penalises high model complexity. As an example, a small l in an RBF kernel produces more 'wiggly' functions that overfit to the data. This term hence illustrates the effect of Occam's razor in empirical Bayes.
3. The constant term $-\frac{n}{2}\log 2\pi$ has no effect on optimisation.

Therefore, we see the LML assesses how well a set of hyperparameters explains the observed data by balancing data fit and model complexity. Optimisation algorithms also require the gradient, which is given by

$$\frac{\partial}{\partial\theta_j}\log p\left(\boldsymbol{y} \mid X, \boldsymbol{\theta}\right) = \frac{1}{2}\mathrm{Tr}\left((\boldsymbol{\alpha}\boldsymbol{\alpha}^T - \tilde{K}^{-1})\frac{\partial\tilde{K}}{\partial\theta_j}\right)$$

where $\boldsymbol{\alpha} = \tilde{K}^{-1}\boldsymbol{y}$ and each $\frac{\partial\tilde{K}}{\partial\theta_j}$ depends on the current parameter θ_j.

There are still some considerations to be made when using this empirical Bayes approach. Importantly, the LML still requires inverting $\tilde{K}$ which takes $O(n^3)$ time, and gradient calculations take $O(n^2)$ time for each parameter θ_j. Furthermore, the parameter space is usually high-dimensional and can have multiple local extrema. This is demonstrated in Figure 3. Such a challenge is not unique to Bayesian model selection, of course, but worth noting when we choose complex kernels with large $\boldsymbol{\theta}$. The figure highlights that optimising the LML *does not* mean a solution will fit the data as closely as possible, in the sense that minimised noise can be suboptimal compared to, say, a GP with better generalisation ability.

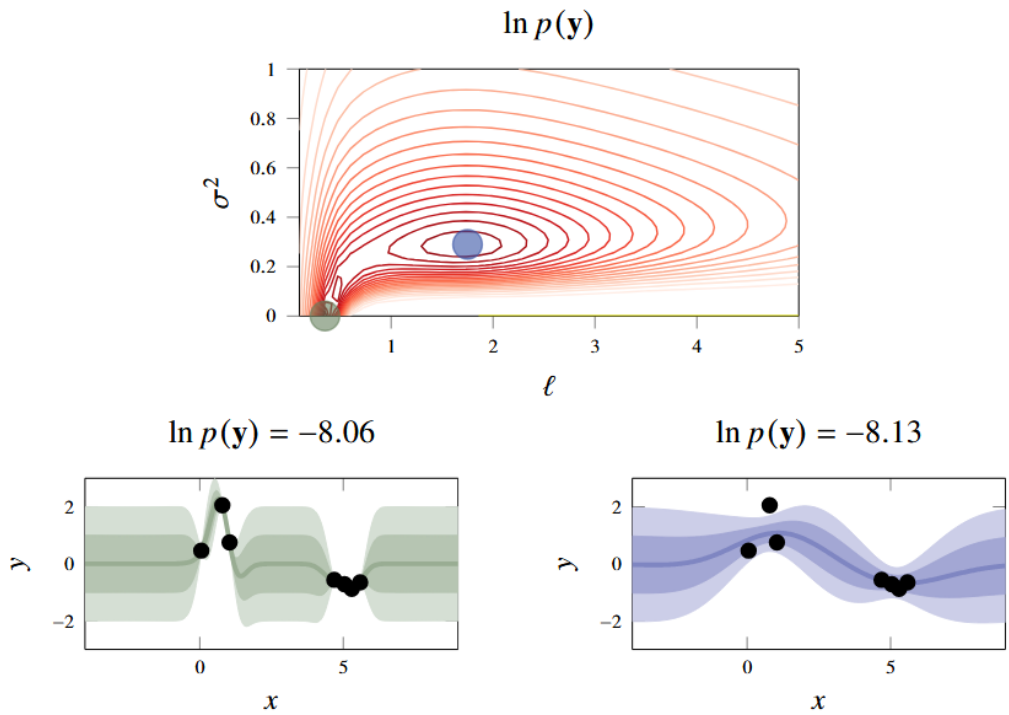

Figure 3: An example of a hyperparameter space with multiple extrema (the two dots in the top panel) for an RBF kernel, where $\boldsymbol{\theta} = (l, \sigma^2)^T$. Each extrema corresponds to a different optimisation result, shown in the bottom panels of corresponding colour. Multiple initialisations, manual assessment of interpretability, or comparison of LML values is recommended for a final model selection. Image taken from [19].

Fortunately, most implementations have marginal likelihood maximisation built-in with their fit functionality. Furthermore, as we will see in Section 4, choice of hyperparameter initialisation $\boldsymbol{\theta}^{(0)}$ does not often require intelligent guesswork when allowing multiple restarts, and hence defaults still converge to good solutions. We shall discuss these code features in the next section.

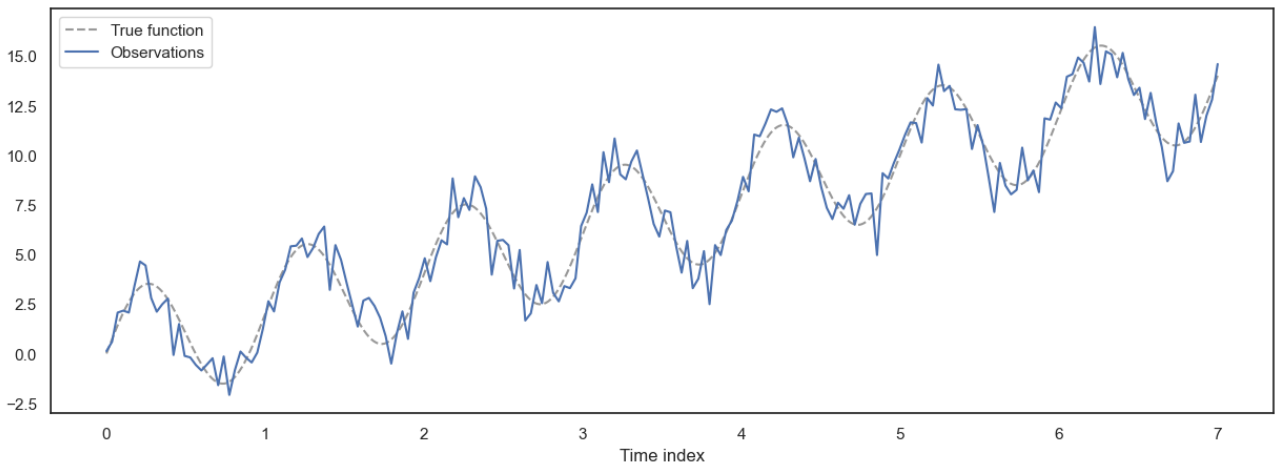

Figure 4: A realisation of size $n = 200$ from the artificial time series (7) with parameters specified. The faint dashed line is the true function (that is, without added noise ε_i).

3 Implementation

3.1 Overview of Datasets

For the demonstration component of this report, we analyse GP models on both an artificial dataset and a real-world dataset. Firstly, the artificial dataset was created to mimic the fundamental behaviour we expect in a time series: trend, seasonality, and noise. As such, we generated observations by the addition of simple components for each, namely a linear function, a sine function, and iid Gaussian noise, respectively. The formula for this function was hence

$$y_i = \beta x_i + \alpha \sin(\omega x_i) + \varepsilon_i \tag{7}$$

where the x_i are just linearly spaced over some predefined domain $[a, b]$ and $\varepsilon_i \sim N(0, \sigma^2)$ for $i = 1, ..., n$. For the dataset featured in this report, we chose a slope of $\beta = 2$; an amplitude of $\alpha = 3$; a frequency of $\omega = 2\pi$; a noise variance of $\sigma^2 = 1$; and $n = 200$ samples over a domain of $[0, 7]$. A realisation with such parameters is plotted in Figure 4, and indeed looks suitable for our purposes.

An authentic meteorological dataset for the project was purchased from OpenWeatherMap [24]. The raw dataset contained hourly readings of numerous weather variables from the Brisbane City weather station (ID: 040913). The time period for this dataset spans from `00:00` on January 1, 1979 to `23:00` on September 18, 2024. Overall, the dataset contains over 400,000 observations across 22 features (excluding metadata columns). A feature-based information table can be found in the appendix.

Due to the immense breadth of this raw dataset, some decisions were made in the interest of scope for this report. Firstly, we restrict our analysis to only the mean temperature readings; brief analysis showed that there were no missing values for this feature vector. Secondly, we aggregate the observations to be **weekly mean temperature** rather than hourly. This effectively removes any daily periodicity in the temperature readings, so we can focus mainly on yearly seasonality. Finally, we further restrict the time period to train on a much shorter 10-year period, from Jan 1, 2010 to Dec 31, 2019. This reduces the number of observations to $n = 522$, shown in Figure 5.[8] Despite these restrictions, we maintain that the application of GP regression to this subset will still highlight the successes and limitations of the model in a practical example.

[8]To emphasise these choices, the memory usage of the dataset goes from 86.2 MB to 8.2 KB — a 99.990% size reduction. This is noteworthy upon recalling the $O(n^3)$ time complexity in GPs mentioned earlier. Admittedly, however, we do lose a considerable amount of information; sparse GPs are a clear avenue for future work.

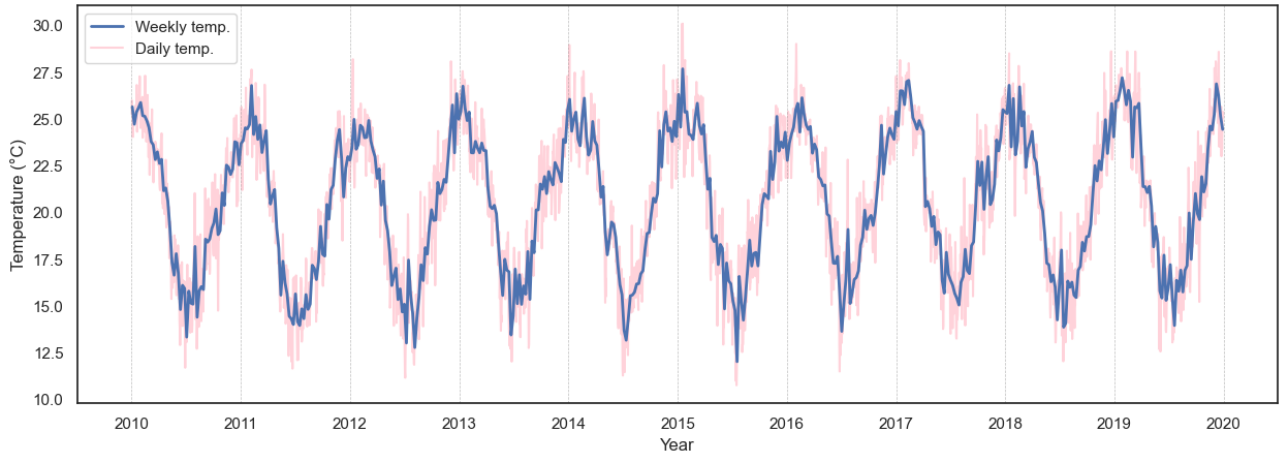

Figure 5: A lineplot of the daily and weekly mean temperature readings for Brisbane over 2010-2020. It is clear that we lose a significant amount of information from the original data, but describing the yearly trends is still an interesting problem. Mean aggregation furthermore has a smoothing effect as can be seen in the plot. Original data sourced from OpenWeatherMap [24].

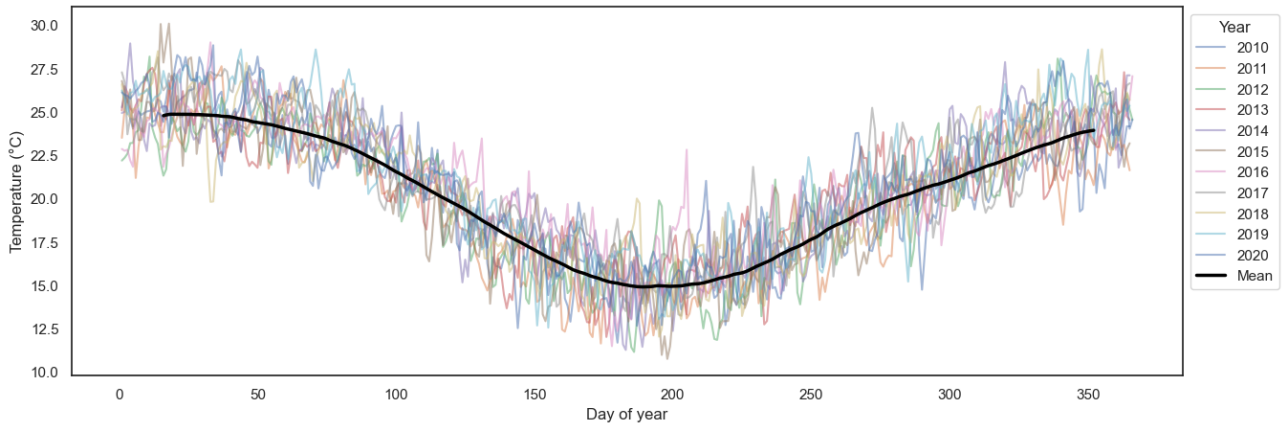

Figure 6: Plot of daily mean temperatures per year over 2010-2020. The thick black line is the average temperature grouped by year, smoothed by a month-long rolling window. This highlights the general yearly trend which we hope to capture using GPs, as well as the magnitude of noise on the daily scale.

3.2 Overview of Code

For this project, we used the Python library *scikit-learn* which implements the necessary tools for modelling with GPs [22]. Another common library for Gaussian processes is *GPy*, recommended by a few of the references in this report [21, 25]. The latter is specifically built for advanced applications of GPs and their extensions, such as sparse GPs and multiple outputs using coregionalisation [15]. However, we opted to use the former due to its familiarity and accessibility.

The general workflow for implementing the GP models is very similar for both datasets. In each script, we first load, format, and preprocess the respective datasets for input into the models. The artificial data is generated by a function with customisable parameters according to (7) and is reproducible using a set seed for RNG. The temperature data was pre-aggregated and saved to a CSV file for ease of access. An important step here is mean-centering the observations based on our prior taking the mean function to be zero by default.

The next step was to define the custom kernels based on the methodology outlined in 2.3. Specifics on parameter initialisation and kernel components for each dataset are discussed and analysed in the next chapter. The GP model object is then instantiated with the respective kernel and fit to the data. The *scikit-learn* implementation is based on Algorithm 2.1 in Rasmussen & Williams (2006) [6], which utilises Cholesky decomposition of the (noisy) Gram matrix. Fortunately, their implementation also automatically optimises the kernel by maximising the LML when the fit function is called. By default, the optimisation is done by the **L-BFGS-B** algorithm, which is relatively fast and a reasonable option since we have gradient information available and boundary conditions on some parameters like $\sigma^2 \geq 0$. Other niceties include options to pass a number of optimiser restarts (with successive runs initialised by log-uniform random samples) and a set seed for reproducibility.

With the kernel optimised and model fitted, the script concludes with model analysis. Fit metrics like the coefficient of determination R^2 are readily available. Most importantly, we plot the posterior mean (MAP estimate) with uncertainty bands ($\sim$95% credible intervals) calculated from the posterior covariance. Forecasting is hence easily visualised as well. Some extra work was necessary to visual the individual kernel components; this was achieved by extracting the component of interest from the fitted model, then 're-fitting' a GP on the component with no optimiser to expose the `predict` method for plotting.

The following snippet shows a very rudimentary example of the modelling workflow:

```python
# Load the data into numpy arrays x and y, mean-centering, etc.

# Fit the GP model
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import *

kernel = ...    # Some intelligent design based on Section 2.3
gp = GaussianProcessRegressor(kernel=kernel, n_restarts_optimizer=3,
    random_state=0)
gp.fit(X, y)

# Model analysis
gp.kernel_      # View the optimised kernel
gp.score(X, y)  # View the R^2 value
# Custom visualisations and so on
```

4 Modelling & Analysis

4.1 Artificial Dataset

Examining the artificial dataset in Figure 4 without knowledge of the true function (7), we see a rising long-term trend and some moderate seasonality, plus some noise. We model each component using an appropriate kernel, then use the sum as the prior covariance in the GP. For the trend, we will not assume this is explicitly linear but rather some smooth trend, and hence use an RBF kernel

$$k_1(x, x') = \zeta_1^2 \exp\left(-\frac{|x - x'|^2}{2\theta_1^2}\right)$$

with scale ζ_1^2 and lengthscale θ_1. For the seasonality, again we will not assume this has perfect cyclicity and hence use the quasi-periodic kernel (5)

$$k_2(x, x') = \zeta_2^2 \exp\left(-\frac{2\sin^2(\pi\,|x - x'|\,/\theta_2)}{\theta_3^2}\right) \exp\left(-\frac{|x - x'|^2}{2\theta_4^2}\right)$$

with scale ζ_2^2, period $\theta_2 = 1$ (fixed), periodic smoothness θ_3, and a decay controlled by θ_4. Lastly, we opt to just model any remaining noise with a white noise kernel

$$k_3(x_i, x_j) = \zeta_3^2 \delta_{ij}$$

with scale ζ_3^2 and where δ_{ij} is the Kronecker delta. Our final composition is thus

$$k_{\boldsymbol{\theta}}(x, x') = k_1(x, x') + k_2(x, x') + k_3(x, x') \tag{8}$$

with unfixed hyperparameters $\boldsymbol{\theta} = (\zeta_1, \zeta_2, \zeta_3, \theta_1, \theta_3, \theta_4)^T$. Initial parameters are just set to defaults, as shown in the snippet below. We additionally have bounds on the noise parameter to ensure an apt fit to the data.

```
kernel_lin = ConstantKernel(1.0) * RBF(1.0)
kernel_per = ConstantKernel(1.0) * ExpSineSquared(length_scale=1.0,
    periodicity=1, periodicity_bounds="fixed") * RBF(1.0)
kernel_noi = WhiteKernel(1.0, noise_level_bounds=(1e-5, 1e1))
kernel = kernel_lin + kernel_per + kernel_noi
```

The results of the GP model on the artificial dataset are plotted in Figure 7. The model appears to explain the data very well, with $R^2 = 0.9544$ and LML value of -292.88. Viewing the optimised kernel, we can analyse the fitted parameters to understand what the model has learned (though there is no real-world interpretation in this case). The trend k_1 has a magnitude of $\hat{\zeta}_1 = 34.6$ and lengthscale $\hat{\theta}_1 = 28.6$. As this lengthscale is large — roughly four times the range of observations $[0, 7]$ — this means the RBF is very smooth and hence models the long-term linearity nicely over the data. However, our choice of an RBF kernel means the trend does eventually decay to zero when extrapolating far from the data, which can be seen in the forecasts in Figure 8.

We see some interesting behaviour when investigating the RBF component (decay) of k_2. Its lengthscale $\hat{\theta}_4 = 47.9$ is very long compared to the data range, which implies that there is effectively no decay in the periodic component over the data. This reflects our knowledge of the true function, since the seasonal component is just a consistent sine function. This affects parameter optimisation, however, since this RBF component is essentially redundant and hence the long-term trend is modelled by two different kernel components. It seems that the optimiser has handled the identifiability issue by splitting the trend between the two kernels, rather than implicitly eliminating one component.

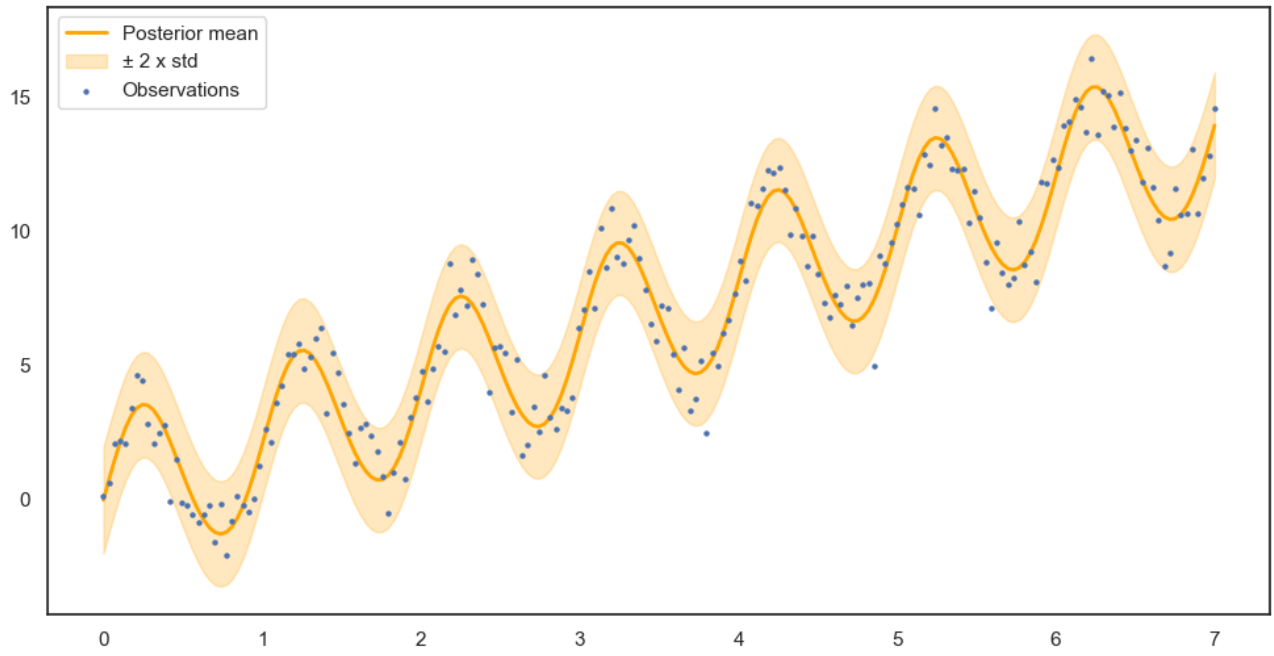

Figure 7: A Gaussian process fit to the artificial dataset with kernel design in (8). The $n = 200$ samples from Figure 4 are included again as markers.

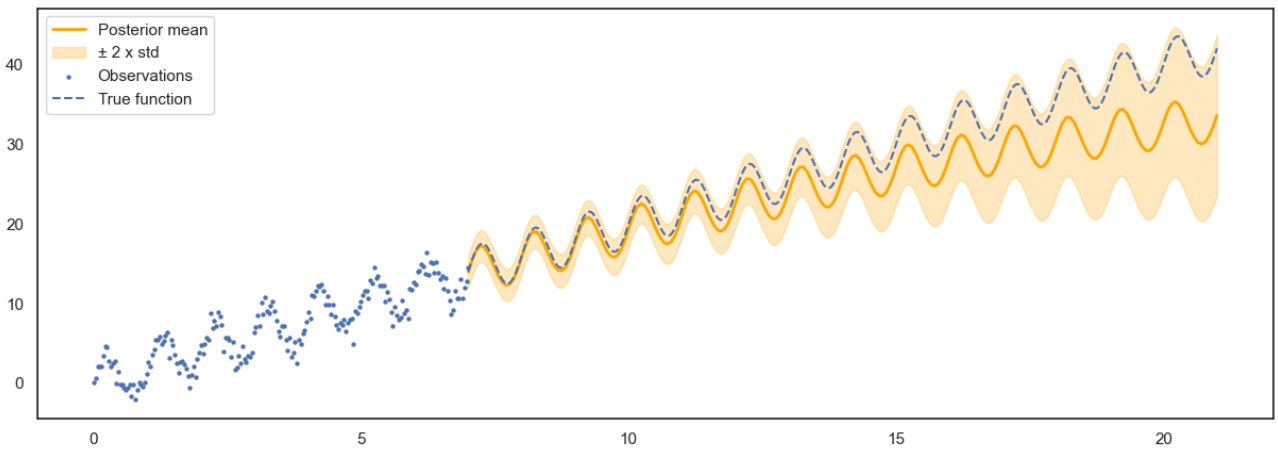

Figure 8: Forecasts of twice the original range of the artificial data. As expected, the uncertainty bands widen as forecast predicts further from the observations. The GP does manage to keep the true function (dashed line) within the uncertainty bands in this plot, however it is clear that the long-term linear behaviour is not captured by our kernel choice, as the latter will eventually decay to zero.

Continuing, we see k_2 had a scale $\hat{\zeta}_2 = 16.7$ and periodic smoothness $\hat{\theta}_3 = 8.6$ which captured the true amplitude well. Finally, the true noise was almost exactly correctly identified with estimated magnitude $\hat{\zeta}_3^2 = 0.915$. We see this reflected in Figures 7 and 8, as the uncertainty bands compactly contain nearly all the observations and they continue to grow steadily with the forecasts. Overall, despite some parameter redundancy, the fit demonstrates the robustness of GPs as the predictive power of the model is still excellent.

4.1.1 Modifying the kernel (known)

Based on the analysis of the redundant decay component, we may wish to amend our choice of kernels and refit the model with an improved understanding. The key problem was that the simple long-term trend was over-parameterised by two flexible RBF components. A reasonable approach would hence remove the decay component from k_2 and just have a purely periodic component. Additionally, we could specify k_1 to be a linear kernel to model the true trend more directly, though this would be harder to justify without good domain knowledge in practice. The linear kernel (or dot-product kernel) is **non-stationary**, unlike the RBF which is a function of $|x - x'|$ and hence stationary, and it has the form

$$k_1(x, x') = \zeta_1^2 \left(\theta_1^2 + x \cdot x'\right) \tag{9}$$

where ζ_1^2 is the scale and θ_1^2 controls the inhomogeneity of the kernel.

```
kernel_lin = ConstantKernel(1.0) * DotProduct(1.0)
kernel_per = ConstantKernel(1.0) * ExpSineSquared(length_scale=1.0,
    periodicity=1, periodicity_bounds="fixed")
kernel_noi = WhiteKernel(1.0, noise_level_bounds=(1e-5, 1e1))
kernel = kernel_lin + kernel_per + kernel_noi
```

The newly fitted GP is shown in Figure 9. The fit gives $R^2 = 0.9538$ which is roughly the same as above, with a very slightly higher LML of -291.26. In practice, either model would be sufficient to explain the data, but the relative simplicity and edge in LML value may make the new model better if the linear kernel is justifiable. For the optimised linear kernel, the parameters were a scale of $\hat{\zeta}_1 = 1.97$ and constant $\hat{\theta}_1 = 0.01$, which indicate this component is homogeneous and accurately captured the true trend slope of $\beta = 2$, as expected. Alternative options to improving the original kernel would be to set bounds on the RBF lengthscales to ensure they model the trends we expected in theory, or to employ automatic methods like ARD as mentioned before.

4.2 Practical Dataset – Mean Weekly Temperature in Brisbane

With a thorough examination of GP regression on the artificial dataset, we now tackle the real-world dataset for this project: weekly mean temperature readings for Brisbane. Based on Figure 5, we again adopt the same general kernel design approach. The long-term trend is modelled by an RBF kernel, though for this time series we notably observe that there doesn't appear an immediately obvious trend.[9] A quasi-periodic kernel is still suitable for the yearly periodicity, and so we shall keep $\theta_2 = 52.2$ weeks as the fixed period.[10] Something new to consider would be a kernel component for any irregular cyclic variations, as mentioned in 2.3, such as a rational quadratic with lengthscales l and α. Finally, despite the smoothing effect of the weekly aggregations, there is clearly still some noise to model; furthermore, it is reasonable to assume that there is some autocorrelation effect which could be modelled by yet another RBF kernel.

[9] Perhaps we expect a very slight positive trend due to effects of climate change, however this would need to be observed on a much longer scale than 10 years. Alternatively, we could look for more extreme events on the hourly or daily interval as climate change increases their frequency. In both cases, however, we'd require sparse GPs for an efficient analysis.

[10] This precision was chosen based on the length of a **tropical year**, which is specifically the time taken to complete a full cycle of the astronomical seasons.

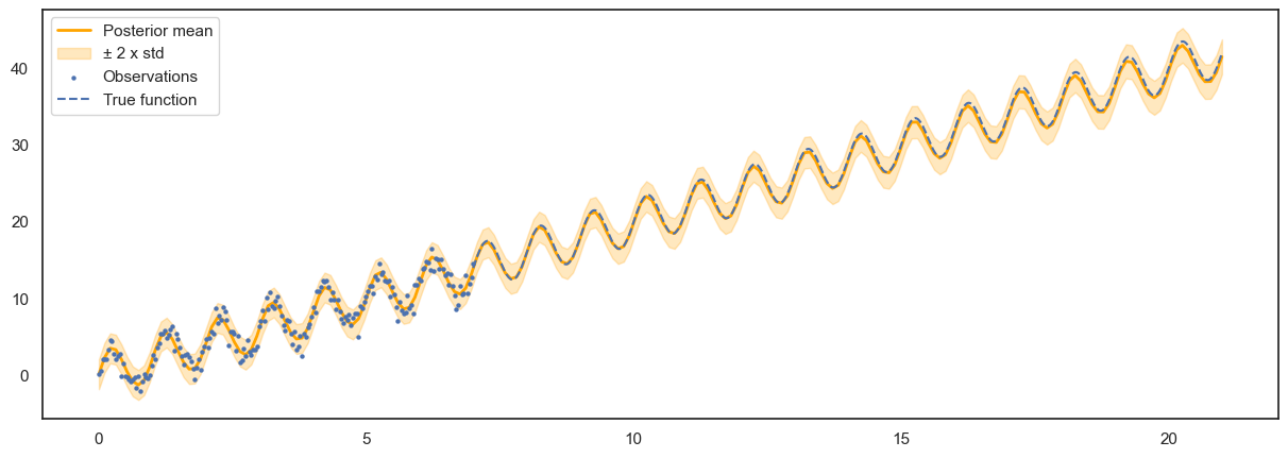

Figure 9: Gaussian process fit on the artificial data with a refined kernel; forecasting again twice the range of samples. We now model the trend directly with a linear kernel (9) and remove the decay from the periodic kernel, essentially matching the true function exactly. This is clearly shown in the forecasts, which now closely follow the true function due to the non-stationarity of the linear kernel.

Overall, our proposed kernel here is

$$k_{\boldsymbol{\theta}}(x, x') = k_1 + k_2 + k_3 + k_4 \tag{10}$$

where

$$\begin{aligned}
k_1(x, x') &= \zeta_1^2 \exp\left(-\frac{|x - x'|^2}{2\theta_1^2}\right) \\
k_2(x, x') &= \zeta_2^2 \exp\left(-\frac{2\sin^2(\pi\,|x - x'|\,/\theta_2)}{\theta_3^2}\right) \exp\left(-\frac{|x - x'|^2}{2\theta_4^2}\right) \\
k_3(x, x') &= \zeta_3^2 \left(1 + \frac{|x - x'|^2}{2\theta_5\theta_6^2}\right)^{-\theta_5} \\
k_4(x_i, x_j) &= \zeta_4^2 \exp\left(-\frac{|x_i - x_j|^2}{2\theta_7^2}\right) + \zeta_5^2 \delta_{ij}
\end{aligned}$$

with hyperparameters $\boldsymbol{\theta} = (\zeta_1, ..., \zeta_5, \theta_1, ..., \theta_7)^T$. This kernel composition is heavily inspired by the *Mauna Loa Atmospheric Carbon Dioxide* regression example from Rasmussen & Williams (2006) [6]. Some parameter initialisation was needed after a few test runs of the optimiser; namely, the RBF lengthscales θ_1, θ_4 were set to be on the same order of magnitude as the range of the data. As there are two independent RBF components in $k_{\boldsymbol{\theta}}$, in theory there is no distinction between which corresponds to noise or signal. For our purposes in this example, the initialisation suggests that k_1 should model the signal (trend).

```
kernel_lin = ConstantKernel(1.0) * RBF(52.0 * 10)
kernel_per = ConstantKernel(1.0) * ExpSineSquared(length_scale=1.0,
    periodicity=52.2, periodicity_bounds="fixed") * RBF(52.0 * 10)
kernel_irr = ConstantKernel(1.0) * RationalQuadratic(length_scale=1.0, alpha
    =1.0)
kernel_noi = ConstantKernel(1.0) * RBF(1.0) + WhiteKernel(1.0,
    noise_level_bounds=(1e-5, 1e1))
kernel = kernel_lin + kernel_per + kernel_irr + kernel_noi
```

The Gaussian process with this complex kernel is shown in Figure 10. This model explains the

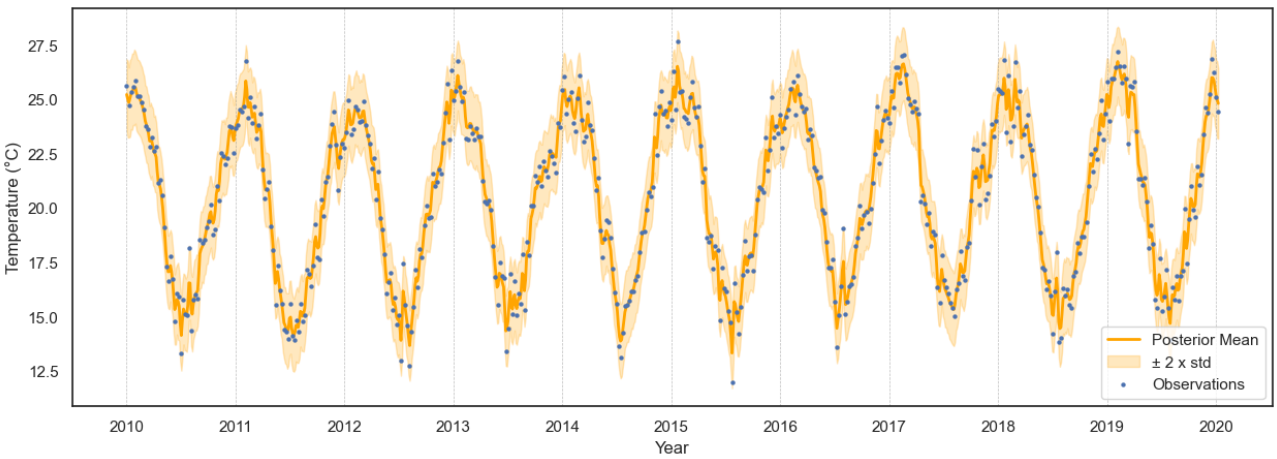

Figure 10: A Gaussian process fit to the temperature dataset with kernel design in (10). The $n = 522$ observations of weekly mean temperature are included as markers.

data extremely well, given $R^2 = 0.9884$ with a LML value of -774.09. As usual, we examine the optimised kernel to understand why this fit is suspiciously good. Firstly, for the long-term trend component k_1 we have $\hat{\zeta}_1 = 2.02°\text{C}$ and $\hat{\theta}_1 = 1.08 \times 10^3$ weeks.[11] This lengthscale is about twice the size of the data range, and so we have a smooth and very weak trend over the data. A plot of this RBF kernel is given in Figure 11 to highlight this gentle trend behaviour. The rational quadratic kernel k_3 has scale $\hat{\zeta}_3 = 0.795°\text{C}$, shape $\hat{\theta}_5 = 0.373$, and length $\hat{\theta}_6 = 0.484$ weeks. These small parameters ($\hat{\theta}_6$ is less than one unit of time in our model) in conjunction with a small scale factor indicates that the model is basically fitting noise rather than any irregular periodicity. In Figure 10 we indeed see that this component causes the posterior to interpolate the data very closely, nearly appearing to overfit. This is also depicted individually in Figure 11, and furthermore we see that this erratic kernel decays swiftly to zero once outside the data range.

Interpreting the quasi-periodic component k_2, we immediately see that the decay-time $\hat{\theta}_4 = 1 \times 10^5$ weeks has hit the default upper bound during optimisation. This strongly suggests that that fit has essentially removed this decay term from the kernel. Of course, this is highly intuitive because the astronomical behaviour of the yearly seasons does not change (at least not significantly within 10 years). Nonetheless, the periodic kernel had the largest scale factor $\hat{\zeta}_2 = 9.53°\text{C}$ with smoothness $\hat{\theta}_3 = 3.38$, corroborating the obvious importance of this piece to the overall model. We wrap up this analysis with the remaining noise components. The correlated noise RBF had scale $\hat{\zeta}_4 = 0.391°\text{C}$ and length $\hat{\theta}_7 = 0.531$ weeks, the latter of which likely implies that there was no autocorrelation as it is again less than one unit of time. The independent noise had magnitude $\zeta_5^2 = 0.402°\text{C}^2$ and hence the overall noise in the model is roughly no greater than $\sqrt{\hat{\zeta}_3^2 + \hat{\zeta}_4^2 + \hat{\zeta}_5^2} \lesssim 1.1°\text{C}$. Overall, while the fit does still explain the data well, it seems the kernel was slightly over-engineered, with some components clearly unnecessary for modelling our aggregated temperature readings.

4.2.1 Simplifying the kernel (unknown)

Based on our analysis of the previous GP model, there are many redundant components of the kernel (10) which may lead to a simplified fit with better generalisability. Despite that we don't know the true mean temperature function for this real data, we look at Figure 12 to motivate some further kernel exploration regardless. The plot shows that the 3-year GP forecast (2020-2023) disagrees somewhat with the future temperature readings in reality. The true observations appear lower on average compared to the posterior; this is confirmed by the R-squared value

[11] For reference, the range of the training samples (2010-2019) is 520 weeks.

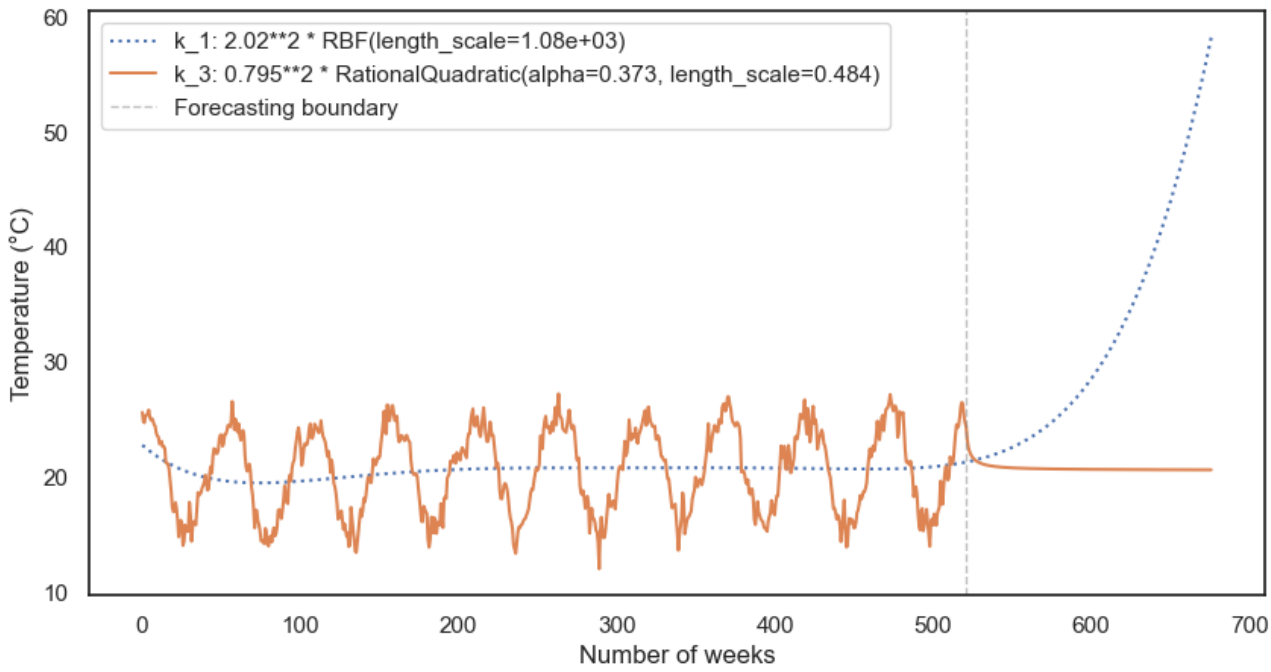

Figure 11: Plot of the optimal kernel components k_1 and k_3 from (10). This provides a better visual interpretation of their respective tuned hyperparameters. We see the RBF (dotted line) is very smooth and confirms there is a very weak long-term trend. Furthermore, the rational quadratic (full line) does not appear to describe any 'medium-term trends' but rather just noise.

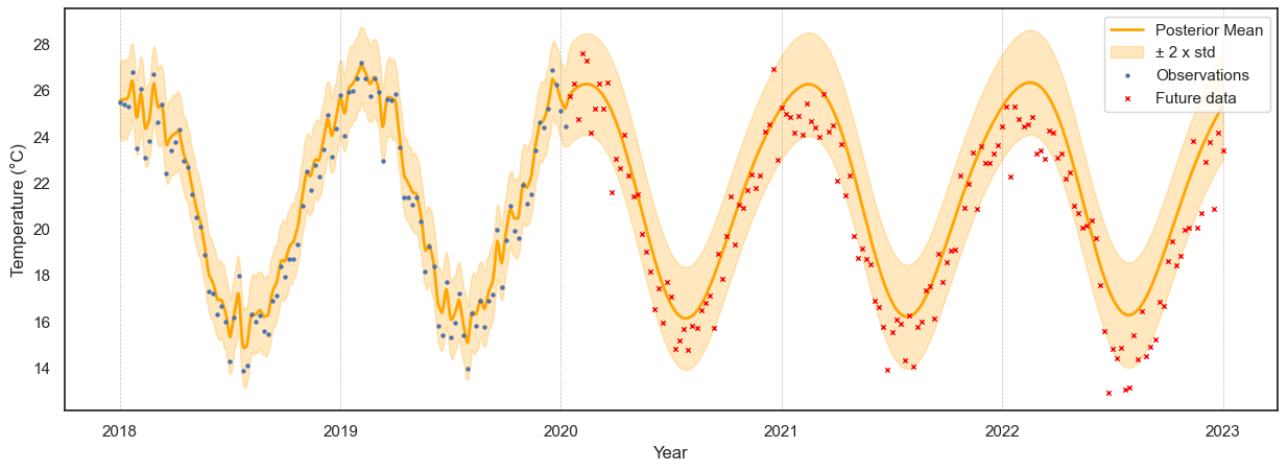

Figure 12: The temperature GP model, with training data (shown 2018-2020) and a 3-year forecast (2020-2023). The plot highlights the change in uncertainty band shape due to the effect of the rational quadratic within the data range. Predictions are much smoother when extrapolating, but in this case the true future data departs from the model, giving poor generalisability.

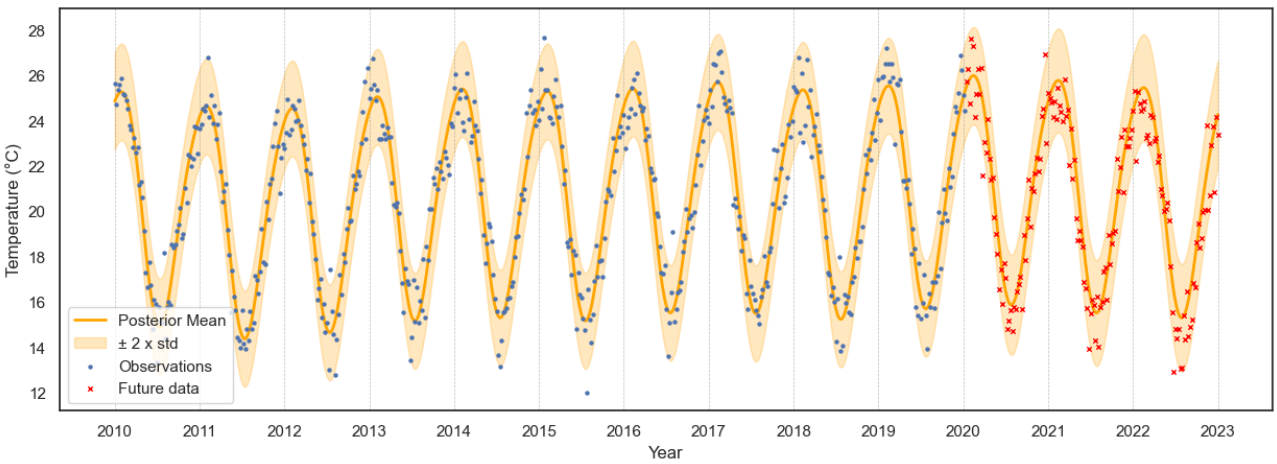

Figure 13: The temperature GP using the simplified kernel (11). Both all training data and the 3-year forecast data are included. The smoothness and improved generalisation are evident in comparison to Figure 12, though the model does not explain the data itself as well without the additional noise kernels.

$R^2 = 0.8503$ calculated for this forecast, which is significantly lower than that on the training data. Furthermore, it seems that even three kernels quantifying noise still could not account for some of these future values, as we see data outside the uncertainty bands like in the troughs.

We hence investigate a much simpler kernel, akin to (8). Inspired by the estimated scale factors, we keep the RBF for the (weak) trend, the periodic (without decay), and the independent noise. This reduces to

$$k_{\boldsymbol{\theta}} = \zeta_1^2 \exp\left(-\frac{|x-x'|^2}{2\theta_1^2}\right) + \zeta_2^2 \exp\left(-\frac{2\sin^2(\pi\,|x-x'|\,/\theta_2)}{\theta_3^2}\right) + \zeta_5^2 \delta_{ij} \tag{11}$$

with $\theta_2 = 52.2$ weeks fixed as before. This kernel is implemented as shown in the snippet below.

```
kernel = ConstantKernel(1.0) * RBF(52.0 * 10) + ConstantKernel(1.0) *
    ExpSineSquared(1.0, periodicity=52.2, periodicity_bounds="fixed") +
    WhiteKernel(1.0)
```

Analysing our new fit results, we see a discouraging $R^2 = 0.9229$ on the training set and lower LML of -789.10. Additionally, the RBF component seems to no longer explain the trend with $\hat{\zeta}_1 = 0.497°\text{C}$ and much smaller lengthscale $\hat{\theta}_1 = 54.8$ weeks. The periodic component is very similar to before, with a large magnitude $\hat{\zeta}_2 = 9.91°\text{C}$ and smoothness $\hat{\theta}_3 = 3.51$. The white noise kernel matches our approximate overall noise calculation from earlier, with $\hat{\zeta}_5^2 = 1.09°\text{C}^2$. Figure 13 shows the smoother fit on the training and future data, and the new GP looks to generalise better in the forecast. Indeed, the R-squared value on the future data is an improved $R^2 = 0.8869$. Therefore, this brief examination of multiple potential fits to the data shows the importance of domain knowledge when selecting kernels and perhaps illustrates the bias-variance tradeoff outside LML maximisation.

4.3 Limitations and Improvements

To conclude this section we discuss the limitations and possible improvements to the Gaussian processes implemented in this project. The previous analyses demonstrated that GPs are flexible, robust models for time series regression and forecasting, but also showed that kernel design and understanding is crucial to ensure good generalisation. On a related note, hyperparameter optimisation is an obvious area for further consideration. For both dataset models, the first kernel attempts not only contained redundant kernel components but LML maximisation often

produced parameters that did not reflect the original intent of their respective kernels. For example, the optimiser split the trend component across the two RBF kernels for the artificial dataset, rather than sending one lengthscale to be very large as in the temperature model. These spurious local maxima can become a problem as the complexity of hyperparameter space increases for more complex kernels. We could test more restarts or other gradient descent algorithms; alternatively, other options for LML maximisation could be examined, such as the aforementioned LOO-CV, hierarchical Bayes, or MCMC.

Kernel design itself is certainly important yet very subjective. The previous modelling required some trial-and-error to produce GPs that balanced data fit and complexity. Perhaps domain knowledge could be improved for the temperature data, utilising other types of kernels like the Matérn family or spectral mixtures for better generalisability. Out-of-the-box methods like ARD [12, 17] have been discussed. In practice, the Bayesian framework allows us to conveniently update the posterior given new observations without refitting the whole model. Multiple-output GPs are another option if we wished to model other correlated meteorological factors.

One limitation of our analysis itself is the lack of other metrics or comparisons to other time series regression models. Specifically, it would be instructive to see statistics comparing performance metrics, fit times, and memory usage of GPs with other popular models like ARIMA and Bayesian structural time series (BSTS). This is particularly relevant to GP applications due to the $O(n^3)$ time complexity and additional memory costs of storing the Gram matrix; our analysis of the mean temperature dataset was limited due to this factor, since the original contained hourly readings. Of course, we have alluded to the use case for sparse GPs throughout this report. These GP approximation methods primarily use a subset $m < n$ of the training data (e.g., inducing points); see Chapter 8 in Rasmussen & Williams (2006) [6] for more details.

5 Conclusion

In this report, we have established the theorectical foundation and practical application of Gaussian processes (GPs) as flexible, non-parametric models. A comprehensive review of the mathematical foundations was given, followed by the formal model definition, the role of covariance functions, and hyperparameter estimation via LML maximisation. Our primary research question centered around time series regression and forecasting with GPs. By employing an additive kernel structure — modelling the standard trend-season-noise decomposition — we successfully analysed both an artificial and real-world dataset. The resulting visualisations demonstrated the excellent fits of the models with intuitive extrapolation, thereby showing the robust predictive power of GPs for forecasting.

Our analysis also identified several practical limitations and directions for future investigation. One such issue was hyperparameter identifiability when optimising complex kernels. Additionally, the most obvious improvement is implementing sparse GPs to overcome the $O(n^3)$ matrix inversion bottleneck on large datasets. Finally, future analysis should focus on extending the framework to more complex real-world challenges, such as dataset with change points, double seasonality, or significant blocks of missing observations.

References

[1] James Mercer. "XVI. Functions of positive and negative type, and their connection the theory of integral equations". In: *Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character* 209.441-458 (1909), pp. 415–446. DOI: 10.1098/rsta.1909.0016. eprint: https://royalsocietypublishing.org/doi/pdf/10.1098/rsta.1909.0016. URL: https://royalsocietypublishing.org/doi/abs/10.1098/rsta.1909.0016.

[2] Christopher Williams and Carl Rasmussen. "Gaussian Processes for Regression". In: *Advances in Neural Information Processing Systems*. Ed. by D. Touretzky, M.C. Mozer, and M. Hasselmo. Vol. 8. MIT Press, 1995. URL: https://proceedings.neurips.cc/paper_files/paper/1995/file/7cce53cf90577442771720a370c3c723-Paper.pdf.

[3] Radford M. Neal. *Bayesian Learning for Neural Networks*. Lecture Notes in Statistics. Springer New York, NY, 1996. URL: https://doi.org/10.1007/978-1-4612-0745-0.

[4] Craig Saunders, Alexander Gammerman, and Volodya Vovk. "Ridge Regression Learning Algorithm in Dual Variables." In: Jan. 1998, pp. 515–521.

[5] R. M. Dudley. *Real Analysis and Probability*. 2nd ed. Cambridge Studies in Advanced Mathematics. Cambridge University Press, 2002.

[6] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. The MIT Press, Nov. 2005. ISBN: 9780262256834. DOI: 10.7551/mitpress/3206.001.0001. eprint: https://direct.mit.edu/book-pdf/2514321/book_9780262256834.pdf. URL: https://doi.org/10.7551/mitpress/3206.001.0001.

[7] Christopher Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006. URL: https://www.microsoft.com/en-us/research/publication/pattern-recognition-machine-learning/.

[8] John B. Conway. *A Course in Functional Analysis*. Graduate Texts in Mathematics. Springer New York, NY, 2010. URL: https://doi.org/10.1007/978-1-4757-4383-8.

[9] Robert C. Rogers Michael Renardy. *An Introduction to Partial Differential Equations*. Texts in Applied Mathematics. Springer New York, NY, 2010. URL: https://doi.org/10.1007/b97427.

[10] Mikhail Lifshits. *Lectures on Gaussian Processes*. SpringerBriefs in Mathematics. Springer Berlin, Heidelberg, 2012. URL: https://doi.org/10.1007/978-3-642-24939-6.

[11] Stephen Roberts et al. "Gaussian processes for time-series modelling". In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 371 (2013). URL: http://doi.org/10.1098/rsta.2011.0550.

[12] David Duvenaud. "Automatic Model Construction with Gaussian Processes". PhD thesis. Computational and Biological Learning Laboratory, University of Cambridge, 2014.

[13] Motonobu Kanagawa et al. *Gaussian Processes and Kernel Methods: A Review on Connections and Equivalences*. 2018. URL: https://arxiv.org/abs/1807.02582.

[14] D.P. Kroese et al. *Data Science and Machine Learning: Mathematical and Statistical Methods*. Chapman & Hall/CRC machine learning & pattern recognition. Boca Raton: CRC Press, 2019. ISBN: 9781138492530. URL: https://people.smp.uq.edu.au/DirkKroese/DSML/.

[15] GPy authors and Sheffield machine learning group. *GPy - A Gaussian Process (GP) framework in Python*. 2020. URL: https://gpy.readthedocs.io/en/latest/.

[16] Rami Krispin. *Decomposition of Time Series*. 2020. URL: https://ramikrispin.github.io/halloween-time-series-workshop/ts-decomposition.html.

[17] Giorgio Corani, Alessio Benavoli, and Marco Zaffalon. "Time Series Forecasting with Gaussian Processes Needs Priors". In: *Machine Learning and Knowledge Discovery in Databases. Applied Data Science Track*. Springer International Publishing, 2021. ISBN: 9783030865146. URL: http://dx.doi.org/10.1007/978-3-030-86514-6_7.

[18] Benyamin Ghojogh et al. *Reproducing Kernel Hilbert Space, Mercer's Theorem, Eigenfunctions, Nyström Method, and Use of Kernels in Machine Learning: Tutorial and Survey*. 2021. arXiv: 2106.08443 [stat.ML]. URL: https://arxiv.org/abs/2106.08443.

[19] Andreas Lindholm et al. *Machine Learning - A First Course for Engineers and Scientists*. Cambridge University Press, 2022. URL: https://smlbook.org.

[20] Kevin P. Murphy. *Probabilistic Machine Learning: An introduction*. MIT Press, 2022. URL: http://probml.github.io/book1.

[21] Jie Wang. "An Intuitive Tutorial to Gaussian Process Regression". In: *Computing in Science & Engineering* 25.4 (2023), pp. 4–11. URL: http://dx.doi.org/10.1109/MCSE.2023.3342149.

[22] scikit-learn contributors. *1.7. Gaussian Processes*. 2025. URL: https://scikit-learn.org/stable/modules/gaussian_process.html.

[23] Wikipedia contributors. *Time series*. 2025. URL: https://en.wikipedia.org/wiki/Time_series.

[24] Open Weather. *Open Weather API Documentation*. https://openweathermap.org/. 2025.

[25] David Duvenaud. *The Kernel Cookbook*. URL: https://www.cs.toronto.edu/~duvenaud/cookbook/.

Appendix

Table 1: Summary of the original dataset containing hourly temperature and weather observations for Brisbane, Australia, recorded in degrees Celsius and related meteorological units from 00:00 on the 1st of January 1979 to 23:00 on the 18th of September 2024.

Feature	**Entries**	**Feature Description**
dt	403,303	Time of data calculation, unix, UTC
dt (ISO)	403,303	Date and time in UTC format
Time Zone	403,303	Shift in seconds from UTC
City Name	403,303	City name
Latitude	403,303	Geographical coordinates of the location (latitude)
Longitude	403,303	Geographical coordinates of the location (longitude)
Temperature	403,303	Temperature in Degrees Celsius
Visibility	327,860	Average visibility, metres (Maximum 10km)
Dew Point	403,303	Atmospheric temperature (Kelvin)
Feels Like	403,303	Human perception of weather
Temperature (Minimum)	403,303	Minimum temperature
Temperature (Maximum)	403,303	Maximum temperature
Pressure	403,303	Atmospheric pressure at sea level (Hectopascal)
Sea Level	0	Atmospheric pressure at sea level (Hectopascal)
Ground Level	0	Atmospheric pressure at ground level (Hectopascal)
Humidity	403,303	Humidity as percentage
Wind Speed	403,303	Wind speed (Meters per Seconds)
Wind Degree	403,303	Wind direction, degrees (meteorological)
Wind Gust	18,952	Wind gust (Meters per Seconds)
Rain (1 hour)	57,459	Rain volume for the last hour (Millimetres)
Rain (3 hours)	26	Rain volume for the last 3 hours (Millimetres)
Snow (1 hour)	0	Snow volume for the last hour, (Millimetres) (liquid state)
Snow (3 hours)	0	Snow volume for the last 3 hours, (Millimetres) (liquid state)
Clouds All	403,303	Cloudiness as percentage
Weather ID	403,303	Weather condition id
Weather Main	403,303	Group of weather parameters (Rain, Snow, Extreme)
Weather Description	403,303	Weather condition within the group
Weather Icon	403,303	Weather icon id